



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления» _____
КАФЕДРА _____ «Программное обеспечение ЭВМ и информационные технологии» _____

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОЙ РАБОТЕ

НА ТЕМУ:

*«Построение реалистического изображения с
учетом оптических свойств поверхностей объектов,
генерации теней и передачи фактуры»*

Студент ИУ7-56Б
(Группа)

(Подпись, дата) В.Б. Барченко
(И.О.Фамилия)

Руководитель курсовой работы

(Подпись, дата) Д.А. Шибанова
(И.О.Фамилия)

2024 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 Аналитический раздел	6
1.1 Формализация объектов синтезируемой сцены	6
1.2 Типы моделей и способы задания поверхностей	7
1.3 Анализ алгоритмов удаления невидимых линий и поверхностей	8
1.3.1 Алгоритм Робертса	8
1.3.2 Алгоритм, использующий z-буфер	10
1.3.3 Алгоритм Варнока	11
1.3.4 Алгоритм обратной трассировки лучей	12
1.3.5 Вывод	13
1.4 Модели освещения	13
1.4.1 Модель освещения Ламберта	14
1.4.2 Модель освещения Фонга	15
1.5 Требования к программному обеспечению	16
1.6 Вывод	16
2 Конструкторский раздел	17
2.1 Геометрические преобразования	17
2.2 Алгоритм обратной трассировки лучей	18
2.3 Пересечение луча с прямоугольным параллелепипедом	19
2.4 Алгоритм Моллера–Трумбора	21
2.5 Вывод	22
3 Технологический раздел	24
3.1 Средства реализации программного обеспечения	24
3.2 Структура программного обеспечения	24
3.3 Реализация алгоритмов	25
3.4 Тестирование	28
3.5 Демонстрация работы программы	29
3.6 Вывод	30

4	Исследовательский раздел	32
4.1	Технические характеристики	32
4.2	Время выполнения реализации алгоритма	32
4.3	Вывод	35
	ЗАКЛЮЧЕНИЕ	36
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	37

ВВЕДЕНИЕ

Появление методов синтеза реалистичных изображений, которые основаны на физических принципах взаимодействия света с объектами, расширило область применения компьютерной графики. Разработанные алгоритмы и программные средства вызвали интерес в сферах архитектуры, градостроительства, проектирования систем освещения, на предприятиях автомобильной и авиационной промышленности, а также в игровой индустрии [1].

Компьютерная графика развивается быстрыми темпами, постоянно появляются новые методы и алгоритмы, позволяющие показывать эффекты различной сложности, затрачивая для этого все меньше вычислительных ресурсов. При этом требования к реалистичности изображений и их физической достоверности растут [1].

Актуальность темы обусловлена потребностью в создании реалистичских изображений в различных сферах деятельности [2].

Целью проекта является разработка программного обеспечения для построения реалистического изображения с учетом оптических свойств объектов, теней и фактуры.

Для успешного достижения поставленной цели необходимо решить набор задач:

- проанализировать предметную область и выбрать существующие методы и алгоритмы;
- спроектировать программное обеспечение для построения реалистического изображения;
- реализовать программное обеспечение;
- исследовать характеристики реализованного программного обеспечения.

1 Аналитический раздел

В данном разделе будут формализованы объекты сцены, описаны типы моделей объектов, способы задания поверхностей, алгоритмы удаления невидимых линий и поверхностей, модели освещения, а также будут представлены требования к программному обеспечению.

1.1 Формализация объектов синтезируемой сцены

Сцена представляет собой виртуальное пространство, содержащее следующие категории объектов:

1. Камера — абстракция физической камеры, которая определяет точку обзора сцены. Параметры камеры включают в себя положение в пространстве, угол и точку обзора.
2. Источники света — объекты, имитирующие освещение сцены.

Типы источников света:

- точечный источник света: освещенность поверхности зависит от ее ориентации: чем меньше угол падения лучей на поверхность, тем меньше освещенность [3]. Задается местоположением и интенсивностью;
 - направленный источник света: имеет ориентацию, но не имеет заданного расположения. Лучи, излучаемые таким источником, параллельны друг другу. При использовании такого типа источника будет казаться, что источник света находится очень далеко от объектов сцены [4].
3. Модель — абстрактное представление объекта, который может быть отображен на сцене. В рассматриваемые модели входят следующие геометрические объекты: куб, сфера, пирамида, конус.

1.2 Типы моделей и способы задания поверхностей

Базовые типы пространственных моделей можно условно разделить на три группы [5]:

- каркасные;
- поверхностные;
- твердотельные.

Каркасные модели — «скелетные» модели объектов на основе задания вершин и ребер. Это одна из простейших форм задания моделей. Недостатком данной модели является то, что она не всегда однозначно передает представление об объекте: невозможно отличить видимые грани от невидимых.

Поверхностная модель — это совокупность поверхностей, ограничивающих и определяющих трехмерный объект в пространстве. Такая модель дает возможность изображения сложных криволинейных участков объектов.

Твердотельная модель описывается в терминах того трехмерного объема, который занимает определяемое ею тело. Это самый достоверный метод создания копии реального объекта [5].

Для решения поставленной задачи подойдет поверхностный тип модели. Каркасные модели не имеют заполненных граней, а твердотельные модели несут в себе исчерпывающую информацию, поэтому выбранный вариант оптимален.

Существует несколько способов представления поверхностей, одними из которых являются аналитический способ и с помощью полигональной сетки.

Аналитический способ — способ задания поверхности, при котором она описывается математическим выражением и представляется как геометрическое место точек или линий, удовлетворяющих уравнению $F(x, y, z) = 0$. Данный способ позволяет точно определить форму поверхности в любой точке.

Полигональная сетка является набором полигонов, которые в совокупности образуют «оболочку» объекта. Она задается списком полигонов и информацией о том, куда «обращен» каждый полигон. Направление полигона задается в виде нормали к плоскости его грани [6].

Для задания поверхности была выбрана полигональная сетка, несмотря на то, что некоторые объекты могут быть не точно представлены и лишь аппроксимироваться ею. Преимуществом является возможность в будущем расширить список рассматриваемых объектов, добавив объекты более сложной формы. А информация о направлении полигонов поможет при закрашивании объекта для определения того, сколько света от источника рассеивается на каждой грани.

1.3 Анализ алгоритмов удаления невидимых линий и поверхностей

Программа, претендующая на реалистичную визуализацию объектов сцены, должна корректно справляться с задачей удаления невидимых линий и поверхностей.

Выделяют алгоритмы, работающие в объектном пространстве, и алгоритмы, работающие в пространстве изображения. Первые связаны с физической системой координат, дают высокоточные результаты. Вторые работают с системой координат экрана, где визуализируются объекты, а точность вычислений ограничена разрешающей способностью экрана [7].

1.3.1 Алгоритм Робертса

Алгоритм работает в объектном пространстве и требует, чтобы грани были выпуклыми многоугольниками. Невыпуклые тела должны быть разбиты на выпуклые части. Удаление проходит в три этапа: в качестве примера будет использована сцена, состоящая из двух пересекающихся каркасных параллелепипедов (рисунок 1.1) [8].

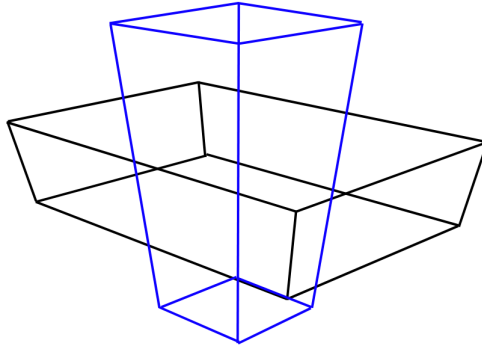


Рисунок 1.1 – Изображение до работы алгоритма Робертса

- На первом этапе каждый объект индивидуально анализируется с целью удаления невидимых граней (рисунок 1.2).

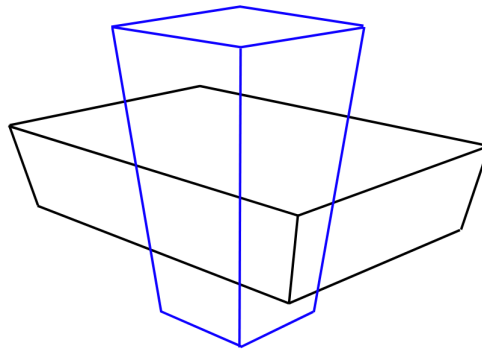


Рисунок 1.2 – Изображение после выполнения первого этапа алгоритма Робертса

- На втором этапе удаляются невидимые ребра, экранируемые другими телами сцены (рисунок 1.3).

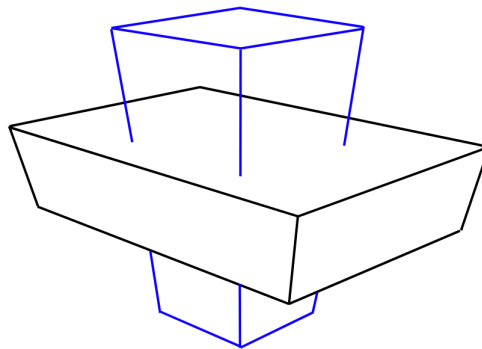


Рисунок 1.3 – Изображение после выполнения второго этапа алгоритма Робертса

- На третьем этапе вычисляются отрезки, которые образуют новые ребра при протыкании телами друг друга (рисунок 1.4).

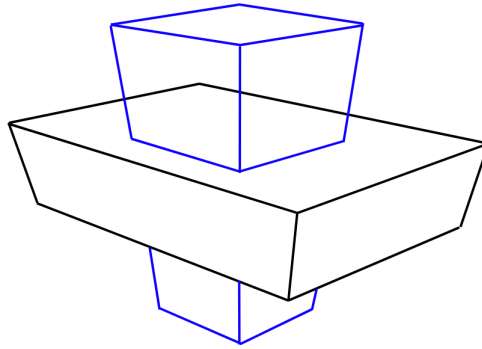


Рисунок 1.4 – Изображение после выполнения третьего этапа алгоритма Робертса

К преимуществам алгоритма Робертса можно отнести высокую точность вычислений [8].

К недостаткам — то, что временные затраты алгоритма пропорциональны квадрату числа граней всех объектов $O(n^2)$ [8]. Для решения этой проблемы можно воспользоваться модификацией алгоритма, например, использовать сортировку вдоль оси z и минимаксные тесты. Еще одним недостатком является реализация проверки объекта на выпуклость и его разбиение на выпуклые составляющие.

1.3.2 Алгоритм, использующий z-буфер

Работает этот алгоритм в пространстве изображения. Для его реализации требуются два буфера: буфер глубины (z-буфер) и буфер кадра. Первый используется для запоминания координаты z , или глубины, каждого видимого пикселя в пространстве изображения (рисунок 1.5). Второй — для хранения информации о цвете каждого пикселя в пространстве изображения. В процессе работы алгоритма глубина каждого нового пикселя, который нужно занести в буфер кадра, сравнивается с глубиной того пикселя, который уже занесен в z-буфер. Если новый пиксел расположен ближе к наблюдателю, чем пиксель, уже находящийся в буфере кадра, то новый пиксель заносится в буфер кадра и производится корректировка z-буфера: в него заносится

глубина нового пикселя. Если же значение глубины нового пикселя меньше, чем хранящееся в буфере, то осуществляется переход к следующей точке [8].

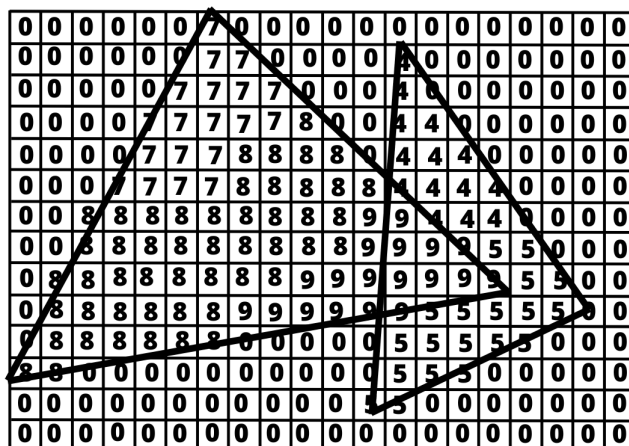


Рисунок 1.5 – Организация z-буфера [8]

Главное преимущество алгоритма z-буфера — простота [8]. Сцены могут быть любой сложности. Объекты заносятся в буфер в произвольном порядке, их не нужно предварительно сортировать. Еще одним преимуществом является линейная зависимость трудоемкости от числа объектов на сцене.

Недостаток алгоритма — большой объем требуемой памяти. Другой недостаток алгоритма z-буфера состоит в трудоемкости и высокой стоимости устранения лестничного эффекта, а также реализации эффектов прозрачности и ряда других задач, которые повышают реалистичность изображения [8].

1.3.3 Алгоритм Варнока

Алгоритм Варнока работает в пространстве изображения. В основе его лежит идея, что большие области изображения однородны, то есть смежные пикселы вдоль обеих осей x и y имеют тенденцию к однородности. Алгоритм рекурсивный, на каждом шаге анализируется видимость граней и, если нельзя легко определить видимость, окно делится на 4 части и анализ повторяется отдельно для каждой из частей. Случаи, когда можно легко определить видимость, следующие (рисунок 1.6) [9]:

1. В окне нет ни одной грани, ничего рисовать не нужно, изображаем фоновым цветом;

2. В окне одна грань, достаточно отобразить эту грань, так как других граней, которые бы могли ее заслонить, нет;
3. Размеры окна равны размеру одного пиксела, выбирается ближайшая грань и ее цветом закрашивается пиксел;
4. Ближайшая грань охватывает все окно, она заслоняет все остальные грани, и достаточно отобразить лишь ее;

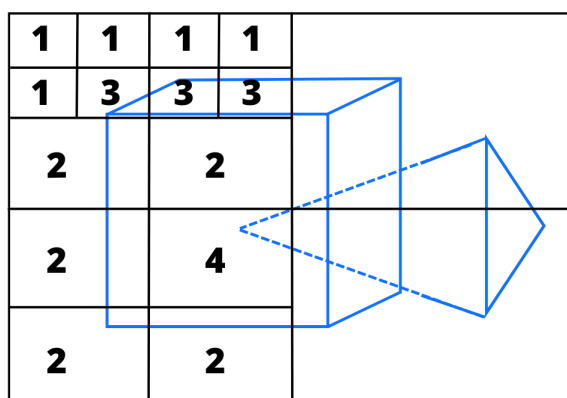


Рисунок 1.6 – Определение видимости алгоритмом Варнока [9]

1.3.4 Алгоритм обратной трассировки лучей

Трассировка лучей – метод реалистической визуализации, моделирующий движение светового луча в изображаемой сцене [6]. Идея данного подхода показана на рисунке 1.7.

Будем рассматривать буфер кадров как массив из пикселов, расположенных в пространстве. Из глаза исходит луч, который через центр пиксела попадает на сцену. Проблема невидимых линий решается автоматически: первая ближайшая поверхность, которую достигает луч, и является видимой, более удаленных поверхностей луч не достигнет. Имея описания источников света на сцене, к точке первого соударения можно применить модель закрашивания, для чего нужно вычислить все компоненты света — фоновый, диффузный и зеркальный. Результирующий цвет затем отображается в данном пикселе. Поскольку траектория луча прослеживается по всей сцене, можно

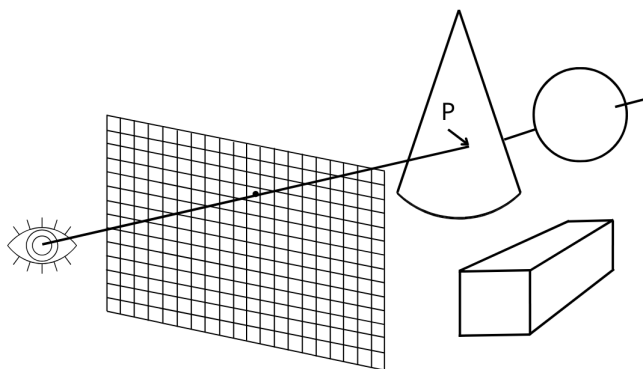


Рисунок 1.7 – Просмотр точки на сцене через пиксел [6]

добавить визуальные эффекты, такие как затенение, отражение и преломление, что приводит к изображениям исключительной реалистичности, которые трудно получить другим способом [6].

1.3.5 Вывод

В ходе анализа алгоритмов для решения поставленной задачи был выбран алгоритм обратной трассировки лучей. Выбор был основан на нескольких факторах, включая качество финальной визуализации и сложности реализации.

1.4 Модели освещения

Модели освещения используются для имитации световых эффектов в компьютерной графике. Свет аппроксимируется на основе физики света. Модели предназначены для вычисления цвета каждого пикселя или количества света, отраженного от различных поверхностей в сцене. Основные факторы, влияющие на цвет точки поверхности, включают [10]:

- свойства источников света;
- способность объектов излучать (отражать, поглощать, пропускать, преломлять) свет;

- взаимодействие с другими объектами (подсветка, затенение, переотражения);
- цвет самой поверхности.

К основным моделям относятся модели Ламберта и Фонга [10].

Компоненты освещения: фоновая компонента закрашивает одним цветом, диффузная закрашивает градиентом цвета, а зеркальная только добавляет пятно блика (рисунок 1.8) [10].

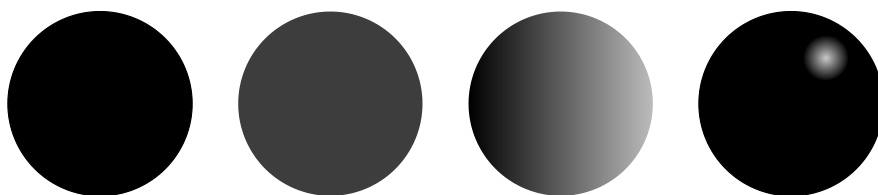


Рисунок 1.8 – Компоненты освещения (слева направо): отсутствуют, фоновая, диффузная и зеркальная [10]

1.4.1 Модель освещения Ламберта

Модель Ламберта — это пример идеально диффузной поверхности, которая одинаково ярко освещается со всех направлений. В природе такая поверхность не встречается, так как в любом материале есть некоторая зеркальная отражающая способность [11].

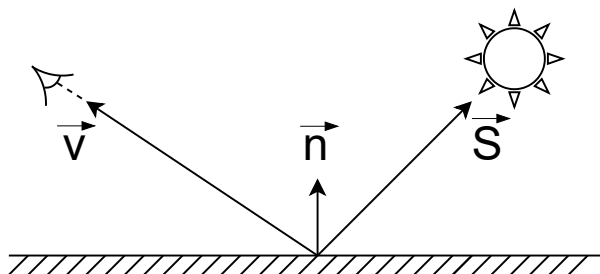


Рисунок 1.9 – Схема для расчета яркости в заданном направлении [11]

Расчет яркости для одного источника света выглядит следующим образом:

$$L = L_i k_d(\vec{s} \cdot \vec{n}), \quad (1.1)$$

где $k_d(\vec{s} \cdot \vec{n})$ — диффузная составляющая, $\vec{s}$ — направление к источнику, $\vec{n}$ — нормаль к поверхности, L_i — интенсивность света от источника.

1.4.2 Модель освещения Фонга

Модель Фонга (рисунок 1.10) добавляет зеркальные блики в модель Ламберта.

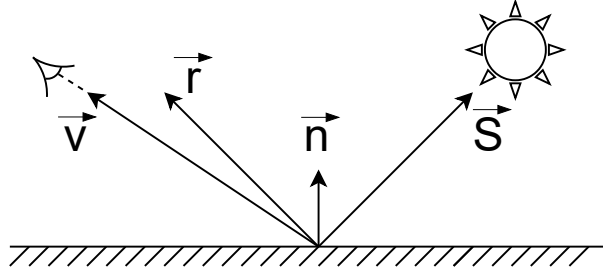


Рисунок 1.10 – Схема отражения для модели Фонга [11]

Полный расчет с помощью модели Фонга [11]:

$$L = k_a L_a + L_i (k_d(\vec{s} \cdot \vec{n}) + k_s(\vec{r} \cdot \vec{v})^{k_e}), \quad (1.2)$$

где $k_a L_a$ — константа, составляющая фонового освещения, $k_s(\vec{r} \cdot \vec{v})^{k_e}$ — зеркальная составляющая, $\vec{r}$ — направление отраженного света.

В отличие от модели Ламберта, модель Фонга учитывает зеркальные отражения (рисунок 1.11), что позволяет создавать блестящие поверхности, такие как металлы или пластик. Поэтому выбор был сделан в ее пользу.

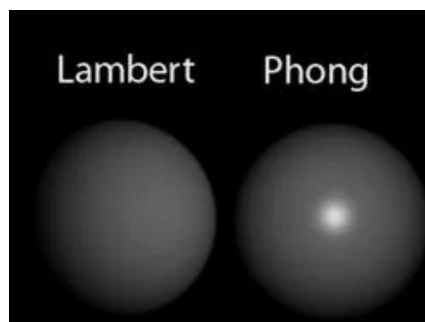


Рисунок 1.11 – Сравнение моделей освещения Ламберта и Фонга

1.5 Требования к программному обеспечению

Программное обеспечение должно работать в двух режимах. В первом режиме необходимо, чтобы программа обеспечивала доступ к следующей функциональности:

- добавление, удаление объектов и источников света;
- поворот, масштабирование, перемещение объектов;
- редактирование параметров (отражение, блеск, прозрачность) объектов;
- изменение положения источника света;
- обзор сцены во всех проекциях (перемещение камеры);

Во втором режиме программное обеспечение должно предоставить возможность построения реалистического изображения и его сохранение.

1.6 Вывод

Выявленные в процессе анализа особенности методов и алгоритмов позволяют сделать вывод о том, что для решения поставленной задачи подходит алгоритм обратной трассировки лучей и модель освещения Фонга, так как это поможет получить реалистическое изображение, учитывающее такие физические факторы, как отражение и преломление.

2 Конструкторский раздел

В данном разделе будут приведены схемы выбранного алгоритма, методы его оптимизации, описаны преобразования объектов.

2.1 Геометрические преобразования

Формула Родригеса используется для поворота вектора в трехмерном пространстве вокруг заданной оси на определенный угол [14]:

$$\mathbf{v}' = \mathbf{v} \cos(\theta) + \mathbf{k} \times \mathbf{v} \sin(\theta) + \mathbf{k}(\mathbf{k} \cdot \mathbf{v})(1 - \cos(\theta)), \quad (2.1)$$

где $\mathbf{v}$ — вектор, который необходимо повернуть, θ — угол поворота в радианах, $\mathbf{k}$ — единичный вектор, определяющий ось вращения.

Формула 2.1 будет применяться для поворотов камеры вокруг осей.

Для вращения моделей сцены на угол θ вокруг осей будут использоваться матрицы поворота:

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) \\ 0 & \sin(\theta) & \cos(\theta) \end{bmatrix}, R_y(\theta) = \begin{bmatrix} \cos(\theta) & 0 & \sin(\theta) \\ 0 & 1 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) \end{bmatrix}, \quad (2.2)$$

$$R_z(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Перенос объекта в трехмерном пространстве достигается путем добавления вектора смещения к каждой точке объекта. Пусть есть точка $P(x, y, z)$ и вектор смещения $T(t_x, t_y, t_z)$, тогда позиция точки после переноса будет вычисляться следующим образом: $P'(x + t_x, y + t_y, z + t_z)$.

Масштабирование объекта в $S(s_x, s_y, s_z)$ раз происходит путем умножения каждой координаты объекта на соответствующий коэффициент масштабирования: $P'(x \cdot s_x, y \cdot s_y, z \cdot s_z)$.

2.2 Алгоритм обратной трассировки лучей

На рисунках 2.1—2.2 представлена схема алгоритма обратной трассировки лучей.

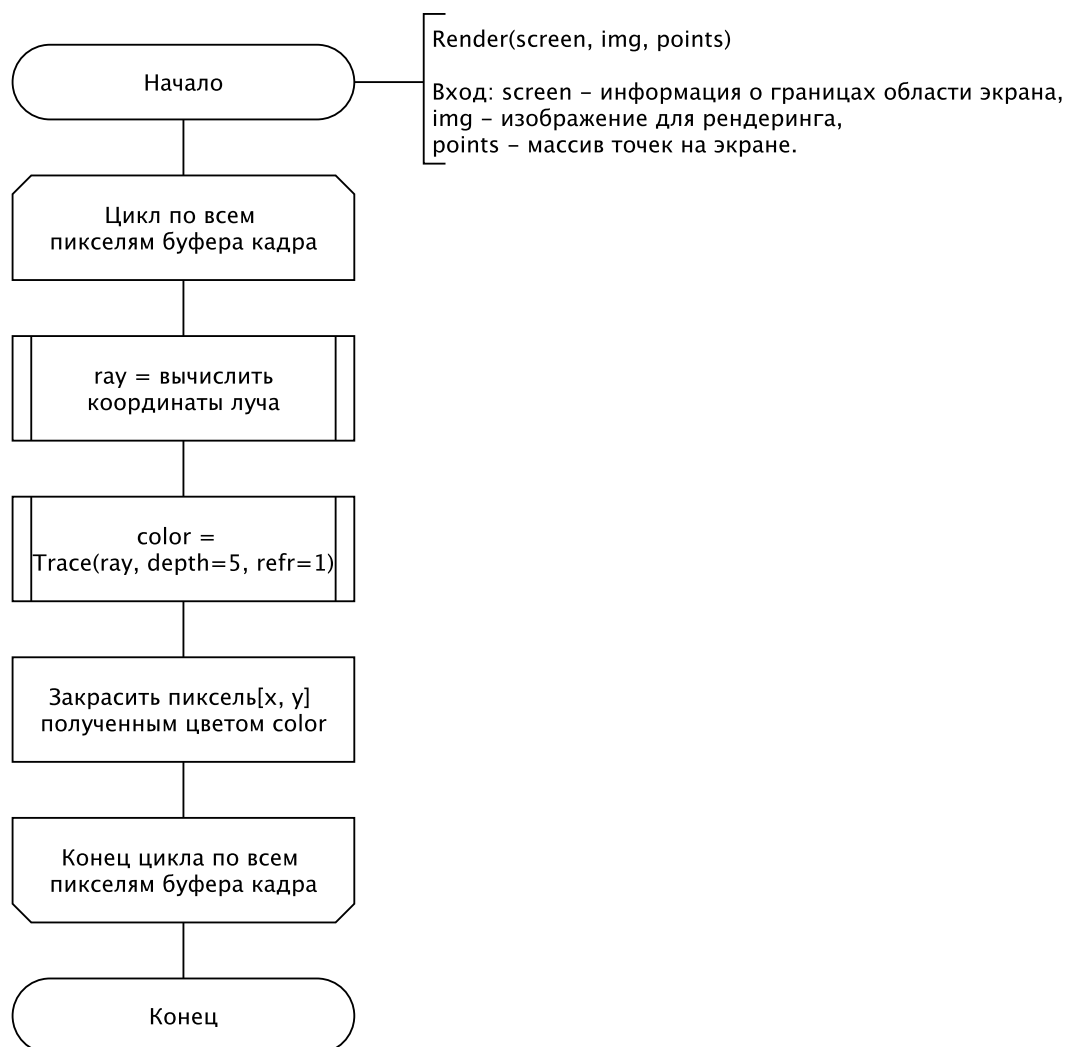


Рисунок 2.1 – Схема алгоритма трассировки лучей

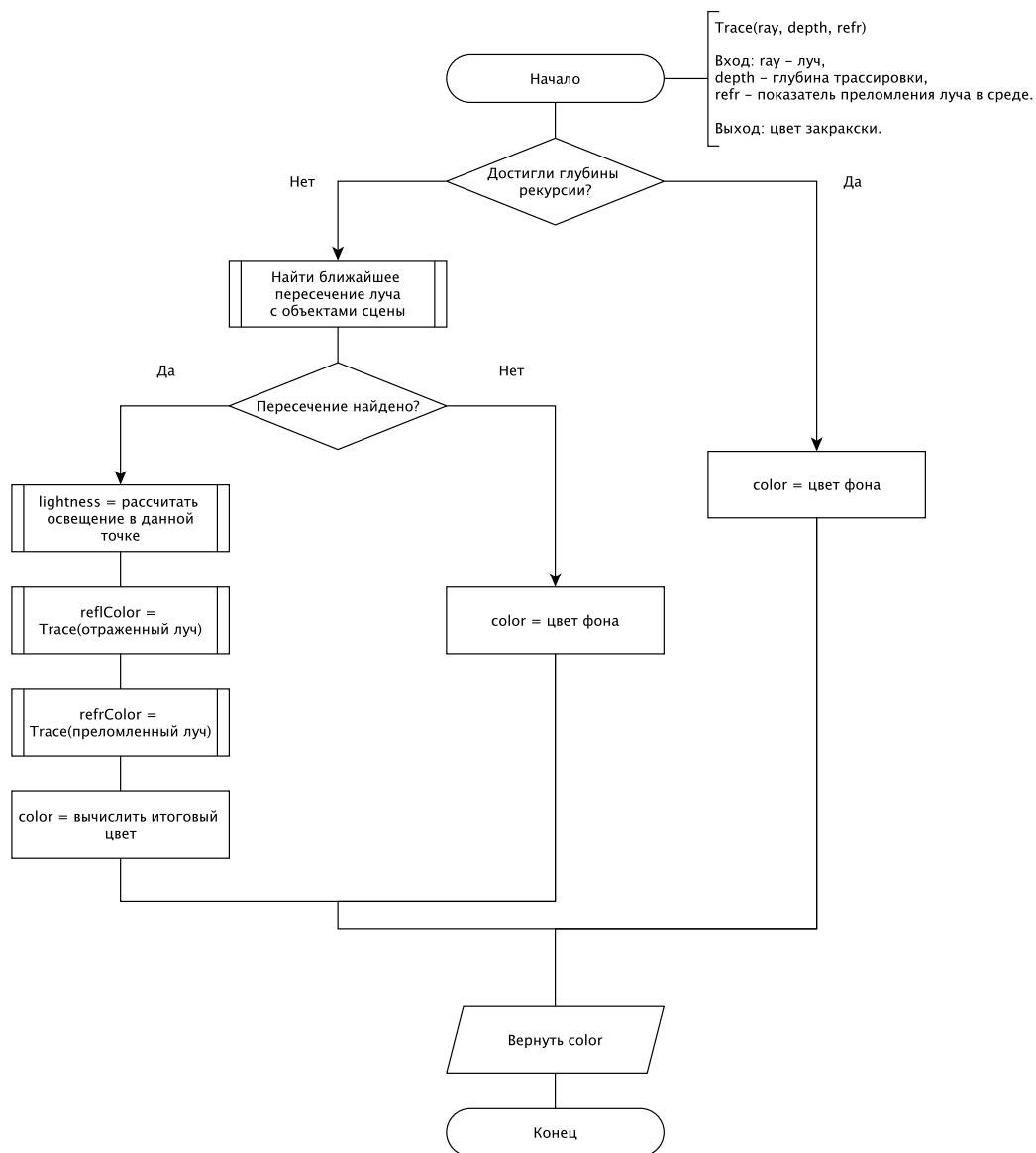


Рисунок 2.2 – Схема вспомогательного рекурсивного метода Trace

2.3 Пересечение луча с прямоугольным параллелепипедом

В методе трассировки лучей до 95% вычислений занимает определение точек пересечения лучей с объектами сцены. Причем большая часть расчетов приходится на лучи, которые не пересекают ни один объект. Поэтому важно минимизировать количество таких лучей, чтобы уменьшить нагрузку на обработку сцены [15].

Для этого объекты сцены заключаются в параллелепипеды. После этого

определяются и отбрасываются лучи, не имеющие общих точек с ними. Этот процесс позволяет исключить лучи, которые не пересекают объекты исходной сцены.

Прямоугольный параллелепипед с параллельными координатным плоскостям сторонами однозначно задается любыми двумя вершинами, примыкающими к одной из его диагоналей. Например, вершинами (x_1, y_1, z_1) и (x_2, y_2, z_2) (рисунок 2.3) [15].

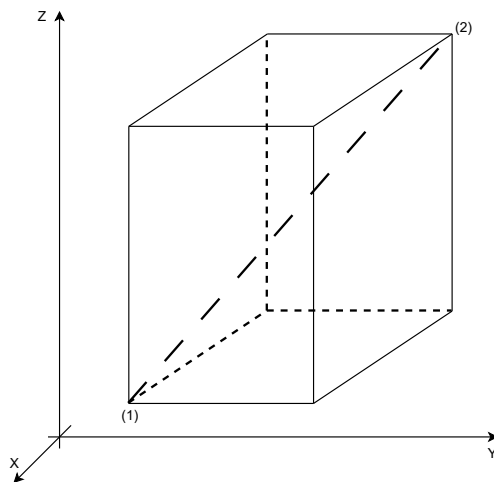


Рисунок 2.3 – Прямоугольный параллелепипед со сторонами, параллельными координатным плоскостям [15]

Возьмем пару плоскостей, параллельных плоскости yz : $x = x_1$ и $x = x_2$. Пусть $\bar{L}$ — луч, исходящий из точки (x_0, y_0, z_0) в направлении вектора (l, m, n) . При l равным нулю, заданный луч параллелен этим плоскостям, и, если $x_0 < x_1$ или $x_0 > x_2$, то он не пересекает рассматриваемый прямоугольный параллелепипед. Если же l не равно 0, то вычисляются отношения 2.3:

$$\begin{aligned} t_{1x} &= \frac{x_1 - x_0}{l}, \\ t_{2x} &= \frac{x_2 - x_0}{l}. \end{aligned} \quad (2.3)$$

Можно считать, что найденные величины связаны неравенством $t_{1x} < t_{2x}$. Пусть $t_n = t_{1x}$, $t_f = t_{2x}$. Считая, что m не равно нулю, и рассматривая вторую пару плоскостей, несущих грани заданного параллелепипеда, $y = y_1$, $y = y_2$, вычисляются величины:

$$\begin{aligned} t_{1y} &= \frac{y_1 - y_0}{m}, \\ t_{2y} &= \frac{y_2 - y_0}{m}. \end{aligned} \tag{2.4}$$

Если $t_{1y} > t_n$, то тогда $t_n = t_{1y}$. Если $t_{2y} < t_f$, то тогда $t_f = t_{2y}$. При $t_n > t_f$ или при $t_f < 0$ заданный луч не пересекает прямоугольный параллелепипед.

Считая, что n не равно нулю, рассматриваем последнюю пару плоскостей: $z = z_1$, $z = z_2$. Находим величины:

$$\begin{aligned} t_{1z} &= \frac{z_1 - z_0}{n}, \\ t_{2z} &= \frac{z_2 - z_0}{n}, \end{aligned} \tag{2.5}$$

и повторяем предыдущие сравнения, приведенные для формулы 2.4.

Если получается, что $0 < t_n < t_f$ или $0 < t_f$, то заданный луч пересечет исходный параллелепипед со сторонами, параллельными координатным осям.

Если луч пересекает прямоугольный параллелепипед (при этом начальная точка луча лежит вне параллелепипеда), то расстояние от начала луча до точки его входа в параллелепипед равно t_n , а до точки выхода луча из параллелепипеда — t_f [15].

2.4 Алгоритм Моллера–Трумбора

Рассмотрим алгоритм Моллера–Трумбора пересечения луча с треугольником в трехмерном пространстве. Одним из преимуществ этого метода является то, что уравнение плоскости, содержащей треугольник, не нужно вычислять, что приводит к значительной экономии памяти [16].

Луч $R(t)$ с началом в точке O и нормализованным направлением D определяется как:

$$R(t) = O + t \cdot D. \tag{2.6}$$

Точка на треугольнике $T(u, v)$ задается формулой:

$$T(u, v) = (1 - u - v) \cdot V_0 + u \cdot V_1 + v \cdot V_2, \quad (2.7)$$

где (u, v) — барицентрические координаты (координаты точки относительно системы координат, с началом в центре тяжести системы [17]), которые должны удовлетворять условиям: $u \geq 0, v \geq 0$ и $u + v \leq 1$. Их можно использовать для отображения текстур, обычной интерполяции, цветовой интерполяции.

Чтобы найти точку пересечения луча $R(t)$ и треугольника $T(u, v)$ нужно решить уравнение $R(t) = T(u, v)$:

$$O + t \cdot D = (1 - u - v) \cdot V_0 + u \cdot V_1 + v \cdot V_2. \quad (2.8)$$

Перегруппируем члены и получим:

$$[-D, V_1 - V_0, V_2 - V_0] \begin{bmatrix} t \\ u \\ v \end{bmatrix} = O - V_0. \quad (2.9)$$

Таким образом, координаты (u, v) и расстояние t от начала луча до точки пересечения можно найти путем решения линейной системы уравнений.

Пусть $E_1 = V_1 - V_0, E_2 = V_2 - V_0$ и $T = O - V_0$, тогда после преобразований можно получить формулу в следующем виде:

$$\begin{bmatrix} t \\ u \\ v \end{bmatrix} = \frac{1}{P \cdot E_1} \begin{bmatrix} Q \cdot E_2 \\ P \cdot T \\ Q \cdot D \end{bmatrix}, \quad (2.10)$$

$$P = (D \times E_2),$$

$$Q = (T \times E_1).$$

2.5 Вывод

В данном разделе были описаны преобразования объектов, приведены схемы выбранного алгоритма и методы его оптимизации: использование параллелепипедов позволит уменьшить количество лучей, которые надо про-

верить на пересечение с объектами сцены, а минимизация вычислений при поиске пересечения луча с треугольником (не нужно вычислять уравнение плоскости треугольника) делает алгоритм пригодным для реализации в реальном времени.

3 Технологический раздел

В данном разделе будут выбраны средства реализации программного обеспечения, описана структура программы, приведены листинги кода.

3.1 Средства реализации программного обеспечения

В данной работе для реализации был выбран язык программирования C Sharp [12]. Данный язык имеет все необходимые и достаточные инструменты для решения поставленной задачи. Для графического интерфейса выбран фреймворк Avalonia [13].

3.2 Структура программного обеспечения

- class Mesh: описывает полигональную сеть для представления объектов;
- class Face: описывает грань полигональной сети (полигон);
- class BoundingBox: содержит информацию о параллелепипеде со сторонами, параллельными координатным осям;
- class Camera: описывает камеру;
- class Light / DirectionLight / PointLight: описывает источник света / направленный источник света / точечный источник света;
- class Transform: содержит матрицы поворота вокруг заданной оси на указанный угол;
- class ILightModel: класс, содержащий метод расчета освещенности в точке сцены;
- class IntersectInfo: содержит информацию о пересечении луча с объектом сцены;

- class ITransformable: описывает методы поворота, масштабирования и переноса;
- class Material: описывает материал для моделирования объектов;
- class MaterialLoader / MeshLoader: описывает загрузчика материалов / загрузчика полигональной сетки из файла;
- class Matrix: описывает матрицу 3x3 и методы работы с ней;
- class Model / ModelConfig: описывает модель объекта в сцене / содержит конфигурационные параметры модели;
- class Ray: описывает луч в трехмерном пространстве;
- class RayTracingStrongShadowsModel: описывает модель освещения для алгоритма трассировки лучей с учетом жестких теней;
- class Scene: представляет собой сцену;
- class Screen: описывает экран для проецирования изображения;
- class Texture / TextureUV: представляет текстуру, описывает текстурные координаты;
- class Tracer: описывает трассировщика лучей;
- class Triangle: описывает треугольник в трехмерном пространстве;
- class Vector3: описывает трехмерный вектор;

3.3 Реализация алгоритмов

На листингах 3.1–3.2 представлена функция трассировщика лучей.

Листинг 3.1 – Функция трассировщика лучей (часть 1)

```

1 Vector3 Trace(Ray ray, int depth, double refr)
2 {
3     if(depth == Depth)
4         return new Vector3();

```

Листинг 3.2 – Функция трассировщика лучей (часть 2)

```
1  var iData = Scene.GetNearestIntersection(ray);
2  if(iData.IsMiss)
3      return new Vector3();
4
5  double in_ray_den = refr;
6  double out_ray_den = 1;
7  if(in_ray_den == 1)
8      out_ray_den = iData.Model!.Material.Refractivity;
9
10 var lightness = LightModel.CalculateLightness(iData.Point!,
11     iData.Normal!, iData.Model!.Material, Scene);
12 var reflColor = new Vector3();
13 if(iData.Model.Material.Reflectivity > 0)
14     reflColor = Trace(ray.Reflect(iData.Point!, iData.Normal!,
15         out_ray_den, in_ray_den), depth + 1, refr);
16
17 var refrColor = new Vector3();
18 if(iData.Model.Material.Transparency > 0) {
19     var refractRay = ray.Refract(iData.Point!, iData.Normal!,
20         out_ray_den, in_ray_den);
21     if(refractRay is not null)
22         refrColor = Trace(refractRay, depth + 1, out_ray_den);
23 }
24
25 return lightness * (1 - iData.Model!.Material.Absorption) *
26     iData.Model!.Material.GetColor(
27         iData.Triangle!.GetTextureUV(iData.Point!))
28     + reflColor * iData.Model.Material.Reflectivity
29     + iData.Model.Material.Transparency * refrColor;
30 }
```

На листингах 3.3–3.4 приведена функция нахождения пересечения с параллелепипедом.

Листинг 3.3 – Функция нахождения пересечения луча с параллелепипедом (часть 1)

```
1 bool Intersect(BoundingBox boundingBox)
2 {
3     double lo = InvDirection.X * (boundingBox.Min.X - Start.X);
4     double hi = InvDirection.X * (boundingBox.Max.X - Start.X);
```

Листинг 3.4 – Функция нахождения пересечения луча с параллелепипедом
(часть 2)

```
1   var tmin = Math.Min(lo , hi);
2   var tmax = Math.Max(lo , hi);
3
4
5   double lo1 = InvDirection.Y*(boundingBox.Min.Y - Start.Y);
6   double hi1 = InvDirection.Y*(boundingBox.Max.Y - Start.Y);
7
8   tmin = Math.Max(tmin , Math.Min(lo1 , hi1));
9   tmax = Math.Min(tmax , Math.Max(lo1 , hi1));
10
11  double lo2 = InvDirection.Z*(boundingBox.Min.Z - Start.Z);
12  double hi2 = InvDirection.Z*(boundingBox.Max.Z - Start.Z);
13
14  tmin = Math.Max(tmin , Math.Min(lo2 , hi2));
15  tmax = Math.Min(tmax , Math.Max(lo2 , hi2));
16
17
18  return (tmin <= tmax) && (tmax > 0.0);
19 }
```

На листингах 3.5–3.6 приведена функция нахождения пересечения с параллелепипедом.

Листинг 3.5 – Функция нахождения пересечения луча с треугольником
(часть 1)

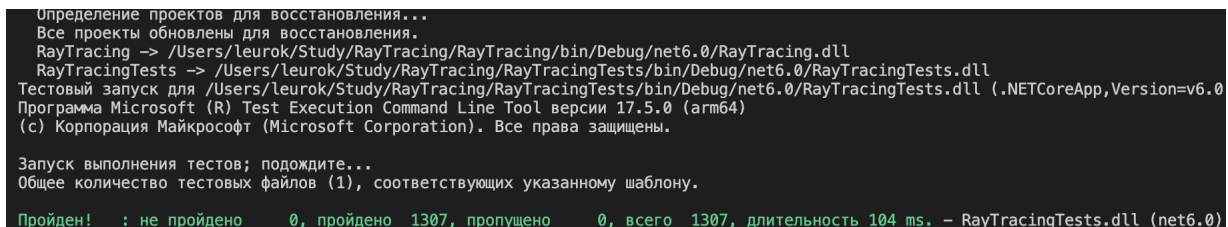
```
1 IntersectInfo Intersect(Ray ray)
2 {
3     var e1 = B - A;
4     var e2 = C - A;
5
6     var pvec = ray.Direction.Cross(e2);
7     var det = e1.Dot(pvec);
8
9     if(Math.Abs(det) < Eps)
10         return new IntersectInfo();
11
12     var inv_det = 1 / det;
13     var tvec = ray.Start - A;
```

Листинг 3.6 – Функция нахождения пересечения луча с треугольником (часть 2)

```
1    var u = tvec.Dot(pvec) * inv_det;
2    if(u < 0 || u > 1)
3        return new IntersectInfo();
4
5    var qvec = tvec.Cross(e1);
6    var v = ray.Direction.Dot(qvec) * inv_det;
7    var distance = e2.Dot(qvec) * inv_det;
8    if(v < 0 || u + v > 1 || distance < 0)
9        return new IntersectInfo();
10
11    var point = ray.Start + ray.Direction * distance;
12
13    return new IntersectInfo {
14        Point = point,
15        Distance = distance,
16        Triangle = this,
17        Normal = GetNormal(point),
18        TextureUV = GetTextureUV(point)
19    };
20 }
```

3.4 Тестирование

Тестирование по методу черного ящика для функций и методов программы было написано с использованием библиотеки NUnit. Все тесты успешно пройдены (рисунок 3.1).



```
Определение проектов для восстановления...
Все проекты обновлены для восстановления.
RayTracing -> /Users/leurok/Study/RayTracing/RayTracing/bin/Debug/net6.0/RayTracing.dll
RayTracingTests -> /Users/leurok/Study/RayTracing/RayTracingTests/bin/Debug/net6.0/RayTracingTests.dll
Тестовый запуск для /Users/leurok/Study/RayTracing/RayTracingTests/bin/Debug/net6.0/RayTracingTests.dll (.NETCoreApp,Version=v6.0)
Программа Microsoft (R) Test Execution Command Line Tool версии 17.5.0 (arm64)
(с) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

Запуск выполнения тестов; подождите...
Общее количество тестовых файлов (1), соответствующих указанному шаблону.

Пройден! : не пройдено 0, пройдено 1307, пропущено 0, всего 1307, длительность 104 ms. - RayTracingTests.dll (net6.0)
```

Рисунок 3.1 – Результат тестирования

3.5 Демонстрация работы программы

На рисунке 3.2 изображен интерфейс программы. Он условно поделен на четыре зоны: в левом верхнем углу находится предпросмотр изображения, в правом верхнем — текущее положение камеры и управление, в левом нижнем — работа с моделью, в правом нижнем — работа с источниками цвета.

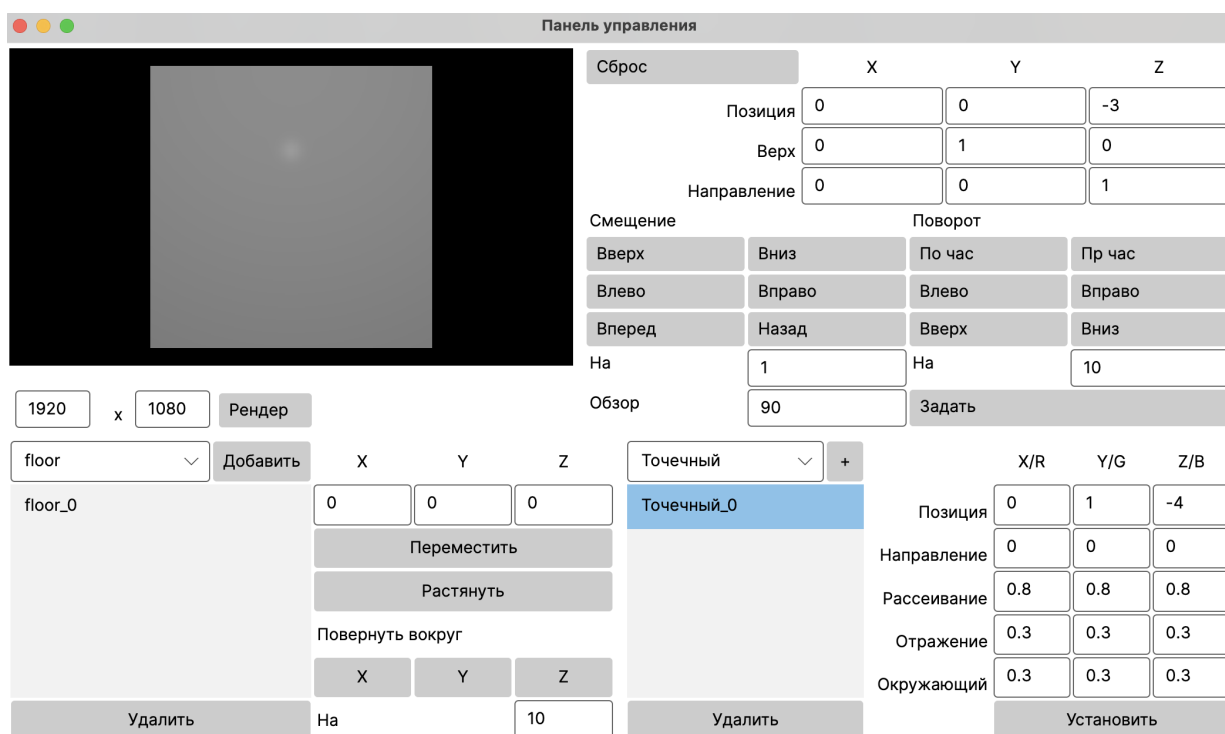


Рисунок 3.2 – Пользовательский интерфейс

На рисунках 3.3–3.4 показаны результаты рендера изображения.

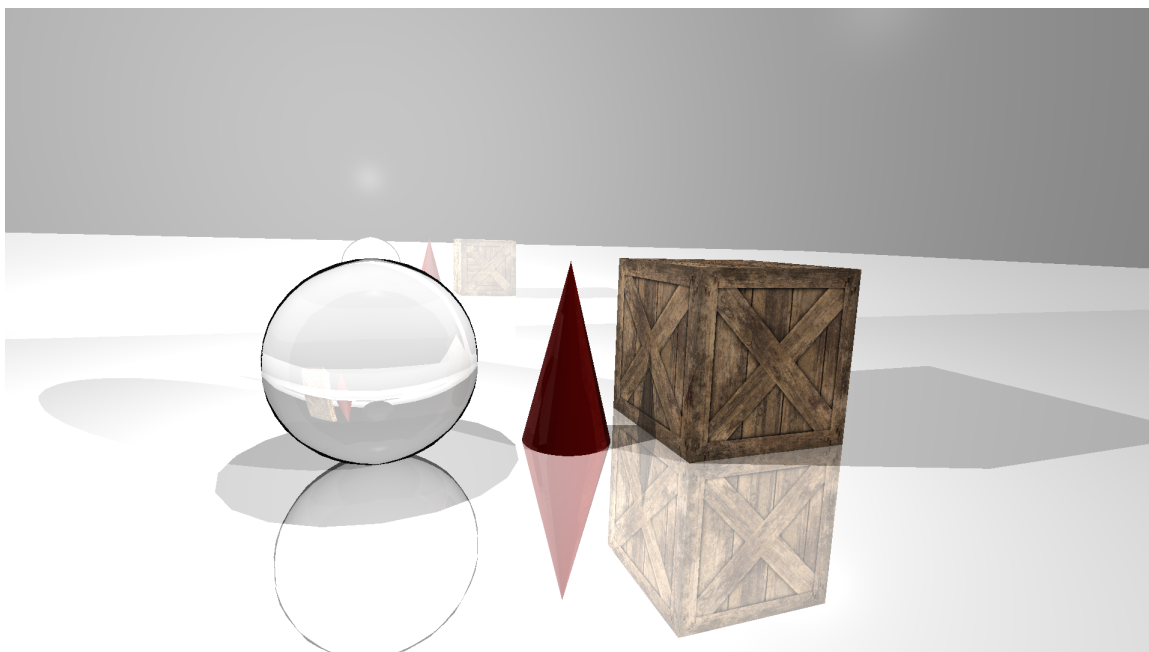


Рисунок 3.3 – Результат рендера изображения (1)

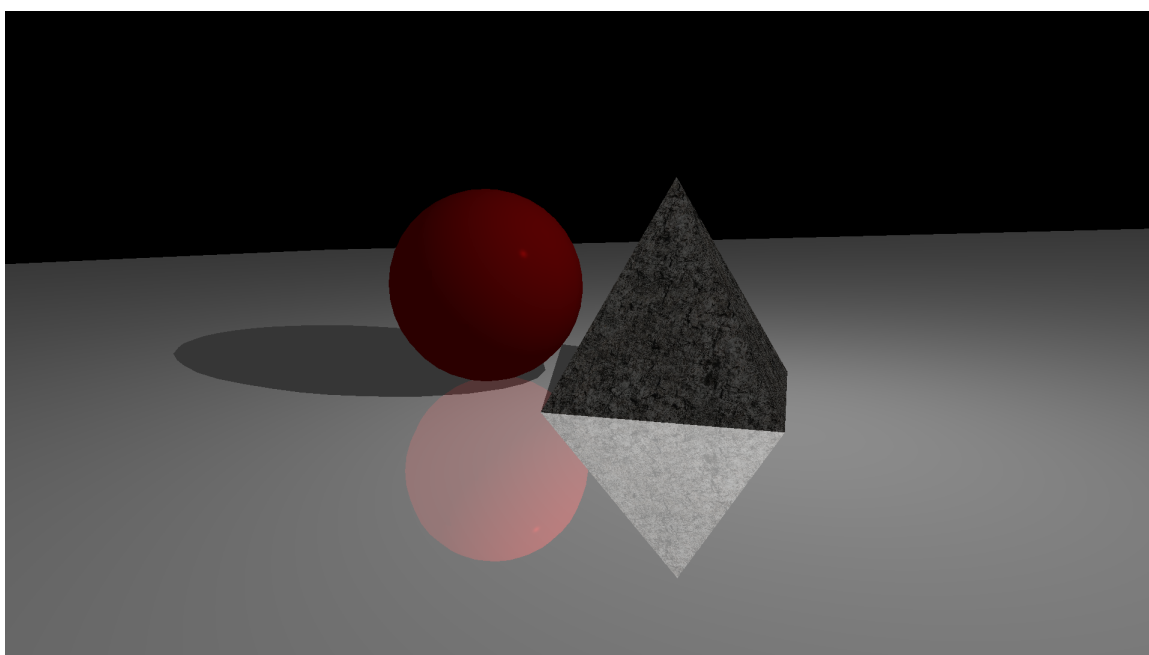


Рисунок 3.4 – Результат рендера изображения (2)

3.6 Вывод

В данном разделе были описаны средства реализации программного обеспечения и листинги кода алгоритмов, рассмотренных в аналитическом и

конструкторском разделах.

4 Исследовательский раздел

В данном разделе будут описаны технические характеристики устройства, будет продемонстрирована работа программного обеспечения и проведено его исследование.

4.1 Технические характеристики

Технические характеристики устройства, на котором выполнялось тестирование:

- операционная система macOS Ventura Версия 13.4 (22F66) [19];
- оперативная память 16 ГБайт;
- процессор Apple M1 Pro [20].

4.2 Время выполнения реализации алгоритма

Для улучшения эффективности программы по времени в программе используются потоки. При делении изображения на части каждый поток может обрабатывать свою часть данных параллельно другим потокам. Это позволяет ускорить обработку данных и сократить общее время выполнения задачи.

Рассмотрим зависимость времени от количества потоков. Сцена содержит стеклянную сферу, куб с текстурой и платформу, на которой они расположены. Результаты исследования приведены в таблице 4.1 и на графике 4.1.

Таблица 4.1 – Зависимость времени работы от количества потоков

Количество потоков	Время (с)
4	102.5150852
16	98.2844692
64	65.3817767
256	63.9542817
400	67.8037844

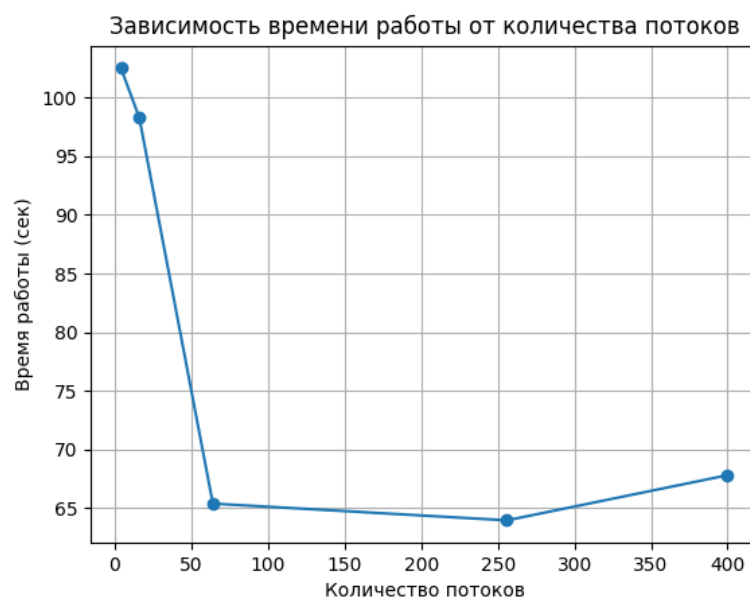


Рисунок 4.1 – Зависимость времени работы от количества потоков

Таким образом, эффективнее по времени использовать 256 потоков.

Исследуем зависимость времени работы реализации алгоритма обратной трассировки лучей от количества объектов в сцене. На сцену добавляется 9 кубов с текстурами и закрашенные цветом, а также направленный источник света. Установлены следующие параметры: 256 потоков, глубина рекурсии 5, коэффициент отражения объекта — 0.1, преломляющая способность объекта — 1.3.

Результаты исследования содержатся в таблицах 4.2–4.3.

Таблица 4.2 – Зависимость времени работы от количества объектов на сцене (цвет)

Количество объектов	Время (с)
1	0.9453964
2	1.2328376
3	1.3683686
4	1.8504181
5	2.2043987
6	2.3590459
7	2.5502223
8	2.9004724
9	3.1002848

Таблица 4.3 – Зависимость времени работы от количества объектов на сцене (текстура)

Количество объектов	Время (с)
1	1.0584696
2	1.3763087
3	1.6467830
4	1.9169437
5	2.2694397
6	2.3958028
7	2.6632753
8	2.9299166
9	3.3796919

На рисунке 4.2 приведен график сравнения зависимости времени работы от закраски цветом или использования текстур.

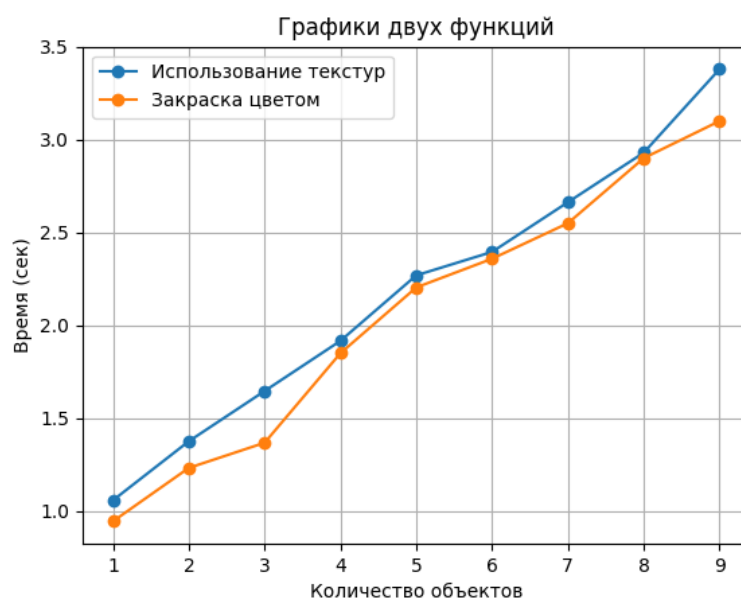


Рисунок 4.2 – Зависимость времени работы от количества объектов на сцене

Закраска цветом происходит быстрее, так как оно не требует дополнительных вычислений для определения цвета каждого пикселя на основе текстурных данных, а также не требует загрузки текстур.

Исследуем зависимость времени от глубины рекурсии реализации алгоритма обратной трассировки лучей. На сцене будет стеклянная сфера, и точечный источник света. Параметры следующие: 256 потоков, коэффициент отражения объекта — 0.05, преломляющая способность объекта — 1.4,

прозрачность объекта — 0.95, поглощающая способность объекта — 0.99.

Результат исследования содержится в таблице 4.4.

Таблица 4.4 – Зависимость времени работы от глубины рекурсии

Количество объектов	Время (с)
1	14.5561826
2	28.7093415
3	43.7881781
4	58.8322416
5	80.1343739

График приведен на рисунке 4.3.

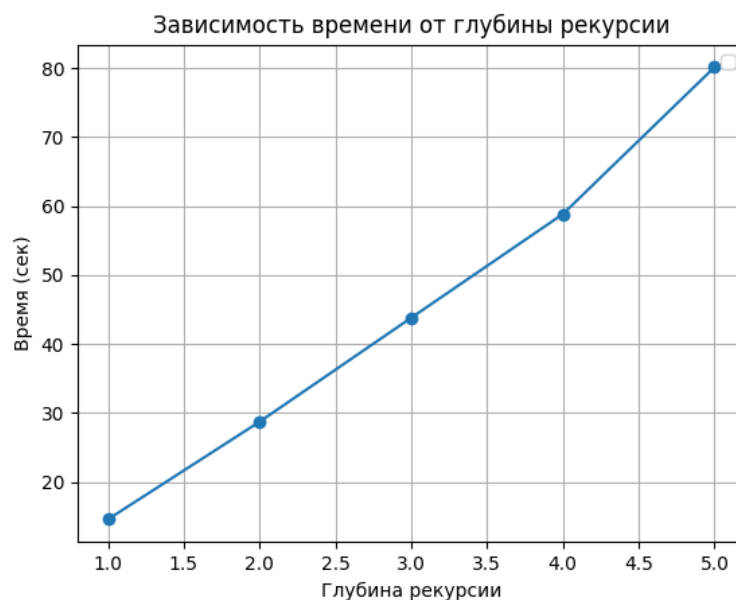


Рисунок 4.3 – Зависимость времени работы от глубины рекурсии

4.3 Вывод

Время работы реализации алгоритма обратной трассировки лучей зависит прежде всего от сложности визуализируемой сцены. Отрисовки девяти кубов цветом эффективнее по времени на 0.9%, чем текстурами. Увеличение глубины рекурсии ведет к линейному увеличению времени. Эффективнее всего использовать 256 потоков.

ЗАКЛЮЧЕНИЕ

Результатом курсового проекта стало программное обеспечение, способное генерировать реалистические изображения трехмерных сцен с использованием алгоритма обратной трассировки лучей. Исследование работы алгоритма обратной трассировки лучей подчеркивает, что его эффективность в значительной степени зависит от сложности сцены, которую необходимо визуализировать. Наблюдается эффективность во времени отрисовки, когда используется простая закрашка цветом вместо текстур. Дополнительное увеличение глубины рекурсии приводит к линейному росту времени выполнения, что следует учитывать при оптимизации производительности. Наиболее эффективным подходом оказывается использование 256 потоков для распараллеливания процесса.

В ходе выполнения данной работы были решены следующие задачи:

- проанализирована предметная область и выбраны существующие методы и алгоритмы;
- спроектировано программное обеспечение для построения реалистического изображения;
- реализовано программное обеспечение;
- исследованы характеристики реализованного программного обеспечения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Волобой А., Галактионов В. Технологии визуализации в задачах современной индустрии. [Электронный ресурс] // Режим доступа: <https://www.sv-journal.org/2013-4/03/index.html> (дата обращения: 28.06.2023)
2. Боголепов Д. К. Методы глобального освещения для интерактивного синтеза изображений сложных сцен на графических процессорах. [Электронный ресурс] // Режим доступа: <https://www.sv-journal.org/2013-4/03/index.html> (дата обращения: 28.06.2023)
3. Тихомиров Ю. Программирование трехмерной графики. — Спб: BHV – Санкт–Петербург, 1998. — 256с.
4. Трухильо С. Программирование графики с использованием Direct3D. Издательство: - ISBN, 1996.
5. Насонова Н. А. К ВОПРОСУ О ПРИМЕНЕНИИ ТВЕРДОТЕЛЬНОГО МОДЕЛИРОВАНИЯ [Электронный ресурс] // Режим доступа: https://elibrary.ru/download/elibrary_21577707_19155404.pdf (дата обращения: 28.06.2023)
6. Хилл Ф. OpenGL. Программирование компьютерной графики. Для профессионалов. — Спб.: Питер. 2002. — 1088с.
7. Роджерс Д. Адамс Дж. Математические основы машинной графики. М.: Мир 1989
8. Переметина Т. О. Компьютерная графика [Электронный ресурс] // Томск: Эль Контент. 2012. Режим доступа: <https://edu.tusur.ru/publications/5613/download> (дата обращения: 28.06.2023.)
9. Польский С. В. Компьютерная графика : учебн.-методич. пособие. – М. : ГОУ ВПО МГУЛ, 2008. – 38 с.
10. Задорожный А.Г. Модели освещения и алгоритмы затенения в компьютерной графике: учебное пособие. – Новосибирск: Изд-во НГТУ, 2020. – 80 с.

11. Фролов В. А. Компьютерная графика. [Электронный ресурс]. Режим доступа: <https://teach-in.ru/file/synopsis/pdf/computer-graphics-frolov-M.pdf> (дата обращения: 20.07.2023).
12. Документация по C#. [Электронный ресурс]. Режим доступа: <https://learn.microsoft.com/ru-ru/dotnet/csharp/>
13. Avalonia UI. [Электронный ресурс]. Режим доступа: <https://www.avaloniaui.net>
14. Johan Ernest Mebius. Derivation of the the Euler-Rodrigues formula for three-dimensional rotations from the general formula for four-dimensional rotations. 2007
15. Шикин Е. В., Боресков А. В. Компьютерная графика. Динамика, реалистические изображения. -М.: ДИАЛОГ- МИФИ, 1996. - 288 с.
16. Moller, Trumbore. Fast, Minimum Storage Ray/Triangle Intersection. Journal of Graphics Tools, 1997.
17. Калматаева Э. В., Митина О. В. Баричентрический метод решения задач
18. Welcome to Python [Электронный ресурс]. Режим доступа: <https://www.python.org> (дата обращения: 20.09.2023).
19. macOS Ventura — Apple(RU). — [Электронный ресурс]. Режим доступа: <https://www.apple.com/macos/ventura/> (дата обращения: 20.09.2023).
20. Процессор M1 [Электронный ресурс] // Режим доступа: <https://www.apple.com/ru/newsroom/2021/10/introducing-m1-pro-and-m1-max-the-most-powerful-chips-apple-has-ever-built/> (дата обращения: 20.09.2023).